

R - Control IP 4 – HTTP/JSON-API

Was ist das?

Die HTTP/JSON-API ist der direkte Draht zum Gerät. Damit können Sie aus eigener Software heraus

- Ausgänge **schalten** und ihren Zustand **abfragen**,
- **Zeitschaltuhren** und **Temperaturaktionen** anlegen und ändern,
- **Temperaturverlauf** und **Ereignisprotokoll** auslesen,
- **Einstellungen** lesen und schreiben,
- den **Gerätestatus** überwachen.

Sie sprechen dieselbe Schnittstelle, die auch die Weboberfläche des Geräts benutzt — alles, was Sie dort sehen, können Sie auch selbst abrufen.

Welche Schnittstelle ist die richtige für mich?

Sie möchten ...	Nehmen Sie
aus eigener Software schalten und konfigurieren	diese API
das Gerät in Home Assistant einbinden	MQTT
eine SPS oder Gebäudeleittechnik anbinden	Modbus TCP
benachrichtigt werden, wenn etwas passiert	Webhook
Zustandsänderungen in Echtzeit verfolgen	WebSocket

In 5 Minuten zum ersten Schaltbefehl

Alle Beispiele nutzen **curl**. Ersetzen Sie **192.168.0.3** durch die Adresse Ihres Geräts.

Schritt 1: Anmelden und Token holen

```
curl -sS -X POST "http://192.168.0.3/api/auth/login" \  
-H "Content-Type: application/json" \  
-d '{"username":"admin","password":"IhrPasswort"}'
```

Antwort:

```
{  
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",  
  "token_type": "Bearer",  
  "expires_in": 1800,  
  "user": { "name": "admin", "permissions":
```

```
[ "controls", "overview", "channels", "schedules", "actions", "settings", "about" ] }
```

Den Wert von **token** brauchen Sie für alle weiteren Aufrufe.

Schritt 2: Token merken

```
TOKEN="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
```

Schritt 3: Ausgang 1 einschalten

```
curl -sS -X POST "http://192.168.0.3/api/relay" \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{"index":0,"command":"on"}'
```

Schritt 4: Zustand abfragen

```
curl -sS "http://192.168.0.3/api/system/status" \
-H "Authorization: Bearer $TOKEN"
```

Das war's. Jeder weitere Aufruf folgt demselben Muster: **URL + Authorization-Kopfzeile**, bei schreibenden Aufrufen zusätzlich JSON im Rumpf.

Grundlagen

Adresse

```
http://<gerät>:<port>/api/...
```

Der Port steht in den Geräteeinstellungen unter **Netzwerk** → **Webserver** (Standard **80**). Ist HTTPS aktiviert, nutzen Sie **https://** und denselben Port.

Statt der IP-Adresse funktioniert meist auch der Hostname, z. B. **http://r-control-ip4.local**.

Anmeldung

Jeder Aufruf außer dem Login braucht die Kopfzeile:

```
Authorization: Bearer <token>
```

Fehlt der Token oder ist er abgelaufen, antwortet das Gerät mit **401** und `{"error":"unauthorized"}`. Holen Sie sich dann einen neuen Token über den Login.

Gültigkeitsdauer: Standardmäßig 30 Minuten. Beim Login können Sie mit `"ttl": 3600` eine andere Dauer anfordern (60 bis 86400 Sekunden). Für Dienste, die dauerhaft laufen, ist es am einfachsten, bei einer 401-Antwort automatisch neu anzumelden.

Schutz gegen Passwortraten

Nach **10 fehlgeschlagenen Anmeldeversuchen** ist der Login **15 Minuten gesperrt** — für alle, geräteweit. Die Antwort lautet dann:

```
HTTP 429 Too Many Requests
Retry-After: 873
{"error":"locked","retry_after_s":873}
```

Prüfen Sie beim Entwickeln Ihre Zugangsdaten sorgfältig; eine Schleife mit falschem Passwort sperrt Sie und alle anderen aus. Den aktuellen Stand können Sie ohne Anmeldung abfragen:

```
curl -sS "http://192.168.0.3/api/auth/status"
# {"locked":false,"retry_after_s":0,"attempts_left":10}
```

Berechtigungen

Jeder Benutzer hat Rechte, die bestimmen, welche Endpunkte er nutzen darf:

Recht	Erlaubt
<code>controls</code>	Ausgänge schalten
<code>overview</code>	Temperaturverlauf, Ereignisprotokoll, Gerätestatus
<code>channels</code>	Ausgänge und deren Einstellungen
<code>schedules</code>	Zeitschaltuhren
<code>actions</code>	Temperaturaktionen
<code>settings</code>	Konfiguration, Benutzer, Firmware, Neustart
<code>about</code>	Geräteinformationen

Fehlt ein Recht, antwortet das Gerät mit **403** und nennt im Text das benötigte Recht. Legen Sie für eigene Software am besten einen eigenen Benutzer an, der nur die Rechte hat, die er wirklich braucht.

Antworten und Fehler

Erfolg ist immer JSON mit HTTP 200:

```
{"status": "ok"}
```

Fehler sehen so aus:

```
{"status": "error", "code": "invalid_request", "message": "Ungültige Eingaben"}
```

HTTP-Code	Bedeutung	Was tun
400	Anfrage fehlerhaft	JSON und Feldnamen prüfen
401	Nicht angemeldet	Neuen Token holen
403	Recht fehlt oder Passwort falsch	Rechte des Benutzers prüfen
404	Nicht gefunden	URL prüfen
409	Konflikt	z. B. Benutzer existiert schon, Limit erreicht
413	Zu groß	Datei- oder Payload-Grenze überschritten
429	Zu viele Versuche	Anmeldesperre, <code>retry_after_s</code> abwarten
500	Gerätefehler	Meldung lesen; ggf. Diagnosedaten exportieren

Schalten

Einen Ausgang schalten

```
POST /api/relay
```

```
{"index": 0, "command": "on"}
```

`index` ist **0-basiert**: 0 = Ausgang 1 ... 3 = Ausgang 4.

<code>command</code>	Wirkung
<code>on</code>	einschalten
<code>off</code>	ausschalten
<code>toggle</code>	umschalten
<code>impulse</code>	kurz schalten, dann zurück (siehe unten)

Impuls — zum Beispiel ein Türöffner für 3 Sekunden:

```
{"index": 0, "command": "impulse", "state": "on", "duration_ms": 3000}
```

Nach Ablauf der Zeit fällt der Ausgang automatisch in den Gegenzustand zurück.

Beispiel: alle vier Ausgänge nacheinander einschalten

```
for i in 0 1 2 3; do
  curl -sS -X POST "http://192.168.0.3/api/relay" \
    -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
    -d "{\"index\":$i,\"command\": \"on\"}"
done
```

Braucht das Recht: controls

Zustand und Überwachung

Gesamtstatus abfragen

```
GET /api/system/status
```

Die wichtigste Abfrage überhaupt: Sie liefert in einem Aufruf Firmware-Version, Laufzeit, Zeitstatus, Temperatur, Zustand aller Schnittstellen, Zähler der Zeitschaltuhren und Sicherheitshinweise.

```
{
  "system": { "app_version": "1.4.0", "uptime_s": 86400, "reset_reason":
"poweron",
              "reset_unexpected": false, "heap_free": 118000, "hostname": "R-
Control-IP4" },
  "time": { "plausible": true, "synced": true, "last_sync_source": "sntp",
"rtc_ok": true },
  "sensor": { "ntc_ok": true, "temperature_c": 21.4 },
  "interfaces": {
    "mqtt": { "enabled": true, "connected": true },
    "modbus": { "enabled": false, "running": false, "port": 502 },
    "webhook": { "enabled": true, "configured": true, "sent_ok": 42,
"sent_failed": 0 }
  },
  "automation": { "scheduler_enabled": true, "schedule_count": 3,
"schedule_limit": 30 },
  "security": { "login_lockout_remaining_s": 0, "admin_uses_default_password":
false }
}
```

Für eine Überwachung reicht es meist, dieses eine Objekt alle 30–60 Sekunden abzuholen.

Braucht das Recht: [overview](#)

Netzwerkstatus

```
GET /api/network/status
```

Liefert Link-Zustand, IP-Adresse, Gateway, DNS, MAC und ob die Adresse per DHCP oder statisch vergeben wurde.

Temperaturverlauf

```
GET /api/telemetry/temperature
```

```
{
  "enabled": true,
  "intervalSeconds": 60,
  "capacity": 1440,
  "samples": [ {"t": 1789720000000, "v": 21.4}, {"t": 1789720060000, "v": 21.5} ]
}
```

t ist der Zeitstempel in Millisekunden, **v** die Temperatur in °C. Das Gerät hält bis zu 1.440 Messpunkte; bei einem Intervall von 60 Sekunden entspricht das 24 Stunden.

Ereignisprotokoll

```
GET /api/overview/actions
```

Die letzten 32 Ereignisse (Schaltvorgänge, Tastendrucke, Systemereignisse), neueste zuerst.

```
{
  "enabled": true,
  "entries": [
    {
      "timestampMs": 1789720000000,
      "type": "output",
      "channel": 2,
      "name": "Hoflicht",
      "action": "on",
      "actionText": "Eingeschaltet"
    }
  ]
}
```

Braucht das Recht: [overview](#)

Zeitschaltuhren

Alle lesen

```
GET /api/scheduler
```

Alle schreiben

```
POST /api/scheduler
```

Wichtig: Der Aufruf ersetzt **alle** Zeitschaltuhren. Lesen Sie erst, ändern Sie das Ergebnis und schreiben Sie es vollständig zurück. Ein einzelner Eintrag lässt sich nicht separat anlegen oder löschen.

Beispiel: eine tägliche Schaltung um 07:00 Uhr

```
{
  "enabled": true,
  "rules": [
    {
      "id": 1789720000000,
      "name": "MorgenEin",
      "type": "daily",
      "time": "07:00:00",
      "time_mode": "fixed",
      "enabled": true,
      "per": [
        ["on", "state", 0],
        ["hold", "state", 0],
        ["hold", "state", 0],
        ["hold", "state", 0]
      ]
    }
  ]
}
```

Das Feld **per** beschreibt, was jeder der vier Ausgänge tun soll — ein Eintrag je Ausgang:

[Aktion, Modus, Impulsdauer in ms]

Position	Mögliche Werte
Aktion	"on" einschalten · "off" ausschalten · "hold" unverändert lassen
Modus	"state" dauerhaft · "impulse" nur kurz
Impulsdauer	Millisekunden, nur bei "impulse"

Wiederholung (**type**):

Wert	Bedeutung	Zusätzliche Felder
once	einmalig	year, month, day

Wert	Bedeutung	Zusätzliche Felder
daily	täglich	–
weekly	wöchentlich	week_days: [1,2,3,4,5] (0 = Sonntag)
monthly	monatlich	day
yearly	jährlich	month, day

Sonnenauf-/untergang statt fester Uhrzeit:

```
{
  "id": 17897200000001,
  "name": "AbendEin",
  "type": "daily",
  "time_mode": "sunset",
  "astro_offset_minutes": -15,
  "astro_not_before": "17:30",
  "astro_not_after": "21:00",
  "enabled": true,
  "per": [
    ["on", "state", 0],
    ["hold", "state", 0],
    ["hold", "state", 0],
    ["hold", "state", 0]
  ]
}
```

Das schaltet 15 Minuten **vor** Sonnenuntergang, frühestens aber um 17:30 und spätestens um 21:00 Uhr. Dafür muss unter **Einstellungen** → **Allgemein** → **Standort** eine Position hinterlegt sein.

Vorschau: Wann läuft eine Regel als Nächstes?

```
GET /api/scheduler/next-runs?id=<Regel-ID>&count=5
```

```
{
  "time_plausible": true,
  "scheduler_enabled": true,
  "runs": [
    {
      "epoch": 1789750800,
      "status": "ok",
      "astro": true,
      "astro_limited": false
    },
    {
      "epoch": 1789837200,
      "status": "skip_holiday",
      "astro": true,
      "astro_limited": true
    }
  ]
}
```

Das Gerät rechnet mit derselben Logik wie im Betrieb. Termine, die wegen Feiertags- oder Ferienregeln übersprungen werden, sind mit **status** enthalten — praktisch zum Prüfen einer Regel, bevor Sie sie scharf schalten.

Braucht das Recht: **schedules**

Temperaturaktionen

```
GET /api/actions
POST /api/actions
```

Auch hier ersetzt der **POST** alle Regeln.

Beispiel: Lüfter einschalten über 25 °C, ausschalten unter 23 °C

```
{
  "enabled": true,
  "rules": [
    {
      "id": 1789720000002,
      "name": "Luefter",
      "enabled": true,
      "condition": { "operator": "above", "threshold": 25.0, "clear_threshold":
23.0 },
      "cooldown_seconds": 60,
      "trigger": { "per": { "ch1": {"act": "on", "mode": "state"} } },
      "recover": { "per": { "ch1": {"act": "off", "mode": "state"} } }
    }
  ]
}
```

Feld	Bedeutung
<code>operator</code>	<code>above</code> = über der Schwelle, <code>below</code> = darunter
<code>threshold</code>	Auslösetemperatur — hier wird die Aktion ausgeführt
<code>clear_threshold</code>	Freigabetemperatur — erst wenn die Temperatur hierher zurückgeht, kann erneut ausgelöst werden
<code>cooldown_seconds</code>	Sperrzeit nach dem Auslösen
<code>trigger</code>	Was beim Auslösen passiert
<code>recover</code>	Was bei der Freigabe passiert (optional)

Die Freigabetemperatur verhindert ständiges Hin- und Herschalten bei schwankender Temperatur. Lassen Sie sie weg, schaltet die Aktion, sobald die Temperatur die Auslöseschwelle wieder unterschreitet — bei einem Wert nahe der Schwelle kann das sehr häufig sein.

Braucht das Recht: `actions`

Einstellungen lesen und schreiben

Gesamte Konfiguration

```
GET /api/config
POST /api/config
```

Der **GET** liefert die vollständige Gerätekonfiguration als JSON, der **POST** ersetzt sie.

Vorsicht: Auch hier gilt „alles oder nichts“. Lesen Sie die Konfiguration, ändern Sie gezielt einzelne Werte und schreiben Sie das Ganze zurück. Schreiben Sie **niemals** ein von Hand zusammengestelltes Teil-JSON — fehlende Abschnitte gehen verloren.

Beispiel: nur den MQTT-Broker ändern (mit **jq**)

```
curl -sS "http://192.168.0.3/api/config" -H "Authorization: Bearer $TOKEN" \
  | jq '.config.interfaces.ha_mqtt.uri = "mqtt://192.168.0.50:1883"' \
  > neu.json

curl -sS -X POST "http://192.168.0.3/api/config" \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  --data-binary @neu.json
```

Sicherung als Datei

```
GET /api/config/bundle    → TAR-Archiv herunterladen
POST /api/config/bundle   → TAR-Archiv einspielen
```

Das Archiv enthält **config.json**, **scheduler.json** und **actions.json**. Ideal, um einen Gerätestand zu sichern oder auf ein zweites Gerät zu übertragen.

```
# Sichern
curl -sS "http://192.168.0.3/api/config/bundle" \
  -H "Authorization: Bearer $TOKEN" -o sicherung.tar

# Wiederherstellen
curl -sS -X POST "http://192.168.0.3/api/config/bundle" \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/x-tar" \
  --data-binary @sicherung.tar
```

Enthält das Bundle Netzwerkänderungen, startet das Gerät neu und ist danach unter der neuen Adresse erreichbar.

Netzwerkeinstellungen übernehmen

Nach dem Ändern von Netzwerkwerten in der Konfiguration:

```
POST /api/network/apply
```

Erst damit werden die neuen Einstellungen aktiv.

Braucht das Recht: `settings`

Weitere nützliche Aufrufe

Aufruf	Zweck
<code>GET /api/about/app</code>	Firmware- und ESP-IDF-Version
<code>POST /api/system/restart</code>	Gerät neu starten
<code>GET /api/system/diagnostics</code>	Diagnose-Archiv (TAR) für den Support
<code>POST /api/interfaces/mqtt/test</code>	MQTT-Broker-Verbindung testen
<code>POST /api/interfaces/webhook/test</code>	Webhook-Ziel testen
<code>POST /api/network/wan-refresh</code>	WAN-IP neu ermitteln
<code>DELETE /api/telemetry/temperature</code>	Temperaturverlauf löschen
<code>DELETE /api/overview/actions</code>	Ereignisprotokoll löschen

Benutzerverwaltung (`GET/POST/PUT/DELETE /api/users`) und Firmware-Update (`POST /api/system/ota`, `GET/POST /api/system/rollback`) sind ebenfalls verfügbar; die Weboberfläche bildet beides vollständig ab.

Ein vollständiges Beispiel

Ein kleines Python-Skript, das sich anmeldet, die Temperatur liest und bei über 25 °C einen Ausgang einschaltet:

```
import requests

GERAET = "http://192.168.0.3"
BENUTZER = "integration"
PASSWORT = "..."

# 1. Anmelden
antwort = requests.post(f"{GERAET}/api/auth/login",
                        json={"username": BENUTZER, "password": PASSWORT},
                        timeout=5)
antwort.raise_for_status()
token = antwort.json()["token"]
kopf = {"Authorization": f"Bearer {token}"}

# 2. Status lesen
status = requests.get(f"{GERAET}/api/system/status", headers=kopf,
```

```
timeout=5).json()
temperatur = status.get("sensor", {}).get("temperature_c")
print(f"Temperatur: {temperatur} °C")

# 3. Bei Bedarf schalten
if temperatur is not None and temperatur > 25.0:
    requests.post(f"{GERAET}/api/relay", headers=kopf,
                  json={"index": 0, "command": "on"}, timeout=5)
    print("Ausgang 1 eingeschaltet")
```

Verwenden Sie für solche Skripte einen **eigenen Benutzer** mit nur den nötigen Rechten (hier: [overview](#) und [controls](#)) — nicht das Admin-Konto.

Fehlersuche

„unauthorized“ bei jedem Aufruf

Fehlt die Kopfzeile `Authorization: Bearer <token>` oder ist der Token abgelaufen? Tokens gelten standardmäßig 30 Minuten. Prüfen Sie auch, ob Sie versehentlich Anführungszeichen mit in den Token kopiert haben.

HTTP 429 beim Login

Die Anmeldesperre ist aktiv — 10 Fehlversuche. `retry_after_s` in der Antwort nennt die verbleibende Zeit. Ein Neustart des Geräts hebt die Sperre ebenfalls auf.

Änderungen an der Konfiguration verschwinden

Sie haben vermutlich ein Teil-JSON geschrieben. `POST /api/config` ersetzt die **gesamte** Konfiguration — immer erst lesen, dann ändern, dann vollständig zurückschreiben.

Zeitschaltuhr läuft nicht

1. Ist die Uhrzeit gestellt? `GET /api/system/status` → `time.plausible` und `time.synced`
2. Ist die Zeitschaltfunktion global aktiv? → `automation.scheduler_enabled`
3. Was sagt die Vorschau? `GET /api/scheduler/next-runs?id=...` zeigt auch übersprungene Termine samt Grund.

Gerät antwortet gar nicht

Prüfen Sie Erreichbarkeit und Port (`ping`, richtiger HTTP-Port in den Einstellungen). Bei aktiviertem HTTPS antwortet das Gerät **nur** noch über `https://` auf demselben Port.

Hinweise für den Produktivbetrieb

- **Eigener Benutzer je Integration**, mit minimalen Rechten. So sehen Sie im Ereignisprotokoll auch, welches System etwas ausgelöst hat.
- **Token wiederverwenden**, nicht vor jedem Aufruf neu anmelden. Bei 401 einmal neu anmelden.

- **Nicht zu häufig abfragen.** Für laufende Überwachung reicht `GET /api/system/status` alle 30–60 Sekunden. Wer Änderungen in Echtzeit braucht, nimmt die WebSocket-Schnittstelle.
- **HTTPS im offenen Netz.** Ohne HTTPS gehen Passwort und Token im Klartext über das Netz. Im abgeschotteten Anlagennetz ist HTTP vertretbar, darüber hinaus nicht.
- **Zeitüberschreitungen einplanen.** Das Gerät ist ein kleiner Steuerrechner; setzen Sie Timeouts von einigen Sekunden und behandeln Sie Ausfälle, statt Aufrufe hart zu blockieren.